

■ CSS Flexbox Layouts and Card Components ■

Grade 4 Coding Class — Detailed Lesson Plan

Learning Objectives

By the end of this session, students will be able to create beautiful card layouts just like the ones they see on their favourite apps!

Objective 01	What is CSS and Flexbox?
Objective 02	Container vs Item Concept
Objective 03	Flexbox Properties & Card Components

Key Properties at a Glance

display: flex Turns Flexbox ON	flex-direction row or column	justify-content Left-Right spacing	align-items Up-Down alignment	gap Space between cards	border-radius Rounded corners
--	--	--	---	-----------------------------------	---

Section 1 — What is CSS?

CSS stands for **Cascading Style Sheets**. It is the language web developers use to add **colours, fonts, sizes, spacing, and layouts** to HTML pages.

■ Analogy

Think of HTML as the **skeleton** of a webpage, and CSS as the **decoration**! Just like a plain white kurta becomes a beautiful festival outfit with colourful embroidery, CSS transforms plain HTML into something amazing!

How CSS Works — Basic Syntax

A CSS rule has two parts: a **selector** (which HTML element to style) and a **declaration block** (the styles inside curly braces).

```
selector {  
  property: value;  
  another-property: another-value;  
}  
  
/* Example */  
.card {  
  background-color: white;  
  border-radius: 12px;  
  padding: 16px;  
}
```

Why Does CSS Matter?

Without CSS, every website would look like a plain text document — just stacked lines of text with no colours or layout. With CSS, the same content becomes a beautiful, usable product like Zomato, Flipkart, or BYJU'S.

Without CSS	With CSS
Messy, stacked text blocks	Clean card-based layout
No colours or images	Rich colours and media
Hard to read or use	Intuitive and beautiful
Like a rough draft	Like a finished product

Section 2 — What is Flexbox?

Flexbox (Flexible Box Layout) is a CSS layout model that makes it easy to arrange elements in **rows or columns**, distribute space between them, and align them — all automatically!

■ Tiffin Box

The **tiffin box (dabba)** is the Flex Container — it holds everything together. Each compartment (rice, dal, vegetables) is a **Flex Item** — neatly organised inside. Flexbox automatically fits items perfectly, just like dabba compartments!

Container vs Item — The Core Concept

Every Flexbox layout has two types of elements working together:

Element	Role	Real-Life Example
Flex Container	The PARENT — gives instructions	Teacher in school assembly
Flex Item	The CHILD — follows instructions	Students obeying the teacher

Activating Flexbox — display: flex

To turn on Flexbox, you simply add **display: flex** to the container element. This is like flipping the ON switch — once activated, all direct children become flex items and automatically line up in a row.

```
.container {  
  display: flex; /* Turns ON Flexbox! */  
}  
  
/* Before: items stack vertically (pile of books on floor) */  
/* After: items line up horizontally (books on a shelf) */
```

■ Remember

display: flex is applied to the **container**, not to the items. The container becomes the 'boss' and all its direct children automatically become flex items.

Section 3 — Flexbox Properties Explained

3.1 flex-direction — Setting the Main Axis

flex-direction controls whether items are arranged in a row (horizontally) or a column (vertically). Think of it as choosing whether students sit in classroom rows or stand in a building's floors.

Property	Value	What It Does
flex-direction	row	Items go LEFT → RIGHT (default). Like classroom seats.
flex-direction	row-reverse	Items go RIGHT → LEFT.
flex-direction	column	Items go TOP → BOTTOM. Like floors in a building.
flex-direction	column-reverse	Items go BOTTOM → TOP.

```
.card-row {
display: flex;
flex-direction: row; /* cards side-by-side */
}

.card-column {
display: flex;
flex-direction: column; /* cards stacked */
}
```

3.2 justify-content — Left-Right Spacing (Main Axis)

justify-content controls how items are spaced along the **main axis**. Imagine arranging food items on a thali plate — where do you put the bowls?

Property	Value	What It Does
justify-content	flex-start	Items crowd to the LEFT (food pushed to one corner).
justify-content	center	Items sit in the MIDDLE (dal bowl at centre of thali).
justify-content	flex-end	Items crowd to the RIGHT.
justify-content	space-between	Equal space BETWEEN items (small bowls spread evenly).
justify-content	space-around	Equal space AROUND each item.
justify-content	space-evenly	Exactly equal space everywhere.

```
.card-row {
display: flex;
justify-content: space-between; /* neat gaps between cards */
}
```

3.3 align-items — Up-Down Alignment (Cross Axis)

align-items controls how items are aligned on the **cross axis** (perpendicular to flex-direction). Think of a bookshelf — do books sit at the top, the middle, the bottom, or are they stretched to fill the shelf?

Property	Value	What It Does
<code>align-items</code>	<code>flex-start</code>	Items align at the TOP (books on the top shelf).
<code>align-items</code>	<code>center</code>	Items align in the MIDDLE vertically – perfectly balanced
<code>align-items</code>	<code>flex-end</code>	Items align at the BOTTOM (books resting on the floor).
<code>align-items</code>	<code>stretch</code>	Items stretch to fill the container height (default).
<code>align-items</code>	<code>baseline</code>	Items align along their text baseline.

```
.card-row {  
display: flex;  
align-items: center; /* all cards vertically centred */  
}
```

Section 4 — Card Components

A **card component** is a self-contained rectangular box that displays a piece of information beautifully — usually with an image, a title, a short description, and sometimes a button.

■ Real Examples

You see cards everywhere! Zomato shows restaurant cards (photo + name + rating + delivery time). Flipkart shows product cards (image + name + price + Add to Cart). BYJU'S shows subject cards (image + class + teacher + Enroll button).

Anatomy of a Card Component

A well-built card has distinct zones, each styled separately:

Zone	HTML Element	Purpose
Image area	div.card-image	Visual that catches the eye first
Title	h3	Name or headline of the item
Description	p	Short supporting text
Button (optional)	button	Call to action (Order, Buy, Enroll)

Section 5 — Building a Card: Step-by-Step

Follow these five build steps to create a professional card component with Flexbox. Each step builds on the last — just like assembling building blocks!

1

HTML Structure — The Card Skeleton

Create the HTML elements that form the card's bones.

```
<!-- Step 1: HTML Structure -->
<div class="card">
  <div class="card-image"></div>
  <h3 class="card-title">Pizza Palace</h3>
  <p class="card-description">Delicious pizzas delivered in 30 min!</p>
</div>
```

2

Card Base CSS — Turning on Flexbox

Style the container and activate Flexbox with flex-direction: column.

```
/* Step 2: Card Base CSS */
.card {
  display: flex; /* Turns ON Flexbox */
  flex-direction: column; /* Stack items top-to-bottom */
  align-items: center; /* Centre everything horizontally */
  width: 200px;
  background-color: white;
  border-radius: 12px; /* Friendly rounded corners */
  box-shadow: 0 2px 8px rgba(0,0,0,0.12);
  overflow: hidden;
}
```

3

Image Placeholder — The Picture Frame

Style the image area to hold the visual that grabs attention first.

```
/* Step 3: Image Placeholder */
.card-image {
  width: 100%; /* Fill the full card width */
  height: 120px; /* Fixed height for the image box */
  background-color: #FFA500; /* Bright orange placeholder colour */
  background-size: cover;
  background-position: center;
}
```

4

Text Styling — Title and Description

Make the card's text readable and attractive.

```
/* Step 4: Text Styling */
.card-title {
  font-size: 16px; /* Readable title size */
  color: #333; /* Dark text */
  margin: 12px 0 4px 0;
```

```

}

.card-description {
font-size: 13px; /* Smaller supporting text */
color: #777; /* Softer grey look */
text-align: center; /* Neat and balanced */
padding: 0 12px 12px;
}

```

5

Multiple Cards in a Row — The Card Grid

Wrap multiple cards in a container and use Flexbox to arrange them.

```

/* Step 5: Card Row Container */
.card-row {
display: flex;
flex-direction: row; /* Cards side by side */
justify-content: space-between; /* Even spacing */
gap: 16px; /* Space between cards */
flex-wrap: wrap; /* Wrap to next line if needed */
padding: 20px;
}

```

```

<!-- Step 5: HTML for the card row -->
<div class="card-row">
<div class="card">...</div>
<div class="card">...</div>
<div class="card">...</div>
</div>

```

Section 6 — Complete Working Example

Here is the complete HTML and CSS combined into one working webpage. Students can type this into a code editor to see a real card layout!

```
<!DOCTYPE html>
<html lang='en'>
<head>
<meta charset='UTF-8'>
<title>My Card Layout</title>
<style>
body {
font-family: Arial, sans-serif;
background: #FFF8F0;
padding: 20px;
}
.card-row {
display: flex;
flex-direction: row;
justify-content: space-between;
gap: 16px;
flex-wrap: wrap;
}
.card {
display: flex;
flex-direction: column;
align-items: center;
width: 200px;
background: white;
border-radius: 12px;
box-shadow: 0 2px 8px rgba(0,0,0,0.12);
overflow: hidden;
}
.card-image {
width: 100%;
height: 120px;
background-color: #FFA500;
}
.card h3 {
font-size: 16px;
color: #333;
margin: 12px 0 4px;
}
.card p {
font-size: 13px;
color: #777;
text-align: center;
padding: 0 12px 12px;
}
</style>
</head>
<body>
<div class='card-row'>
<div class='card'>
```

```
<div class='card-image'></div>
<h3>Pizza Palace</h3>
<p>Delivered in 30 minutes!</p>
</div>
<div class='card'>
<div class='card-image'></div>
<h3>Burger Barn</h3>
<p>Juicy burgers, crispy fries!</p>
</div>
<div class='card'>
<div class='card-image'></div>
<h3>Sushi Spot</h3>
<p>Fresh rolls every day!</p>
</div>
</div>
</body>
</html>
```

Section 7 — Recap Flash Cards

Let's remember the magic tools we learned today! Each row below is a flashcard summarising a key Flexbox concept.

Property / Concept	What It Does	Memory Trick
<code>display: flex</code>	Turns Flexbox ON for the container	The ON Switch!
<code>flex-direction: row</code>	Items go left → right	Students sitting in classroom seats
<code>flex-direction: column</code>	Items go top → bottom	Floors stacked in a building
<code>justify-content: center</code>	Centre items on main axis	Dal bowl at centre of thali
<code>justify-content: space-between</code>	Equal space between items	Bowls spread evenly on thali
<code>align-items: center</code>	Centre items on cross axis	Books sitting at mid-shelf
<code>gap: 16px</code>	Space between flex items	Breathing room between cards
<code>border-radius: 12px</code>	Rounded corners on boxes	Friendly greeting card look
<code>flex-wrap: wrap</code>	Items wrap to next line if no space	Books overflow to next shelf

Section 8 — Homework and Next Session

Homework Assignment

1**Find 3 Real Cards**

Open Zomato, Flipkart, or any app. Find 3 card components. Screenshot or draw them in your notebook.

2**Card Element List**

For each card you found, write down what elements are inside: image, title, rating, price, button, etc.

3**Draw Your Own Card**

Sketch a card for your favourite food, book, or game. Label each zone (image, title, description, button).

Next Session Preview

■ Coming Next

We will make cards **INTERACTIVE**! You will learn CSS hover effects (cards lift up when you mouse over), CSS transitions (smooth animations), and how to make layouts **mobile-friendly** with media queries so they look great on phones too!

Bonus Challenge (Optional)

Can you add a **button** to your card? Try adding this CSS for a hover effect:

```
.card {  
  transition: transform 0.2s; /* Smooth animation */  
}  
  
.card:hover {  
  transform: translateY(-4px); /* Card lifts up on hover! */  
  box-shadow: 0 8px 20px rgba(0,0,0,0.18);  
}
```

■ You Built Your First Flexbox Layout — Great Work! ■

That's what real web developers do every day. Keep practising, keep exploring, and you are on your way to becoming a coding superstar!